

~ -- -zsh

...

...ory_and_test_shell.c

...ano signal_handler.c

~ -- -zsh

~ -- append_stderr

UW PICO 5.09

File: export_state.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <unistd.h>
#include <sys/wait.h>
#include <limits.h>

#define MAX_HIST 100
#define BUF_SIZE 512
#define STATE_FILE "shell_state.txt"

char *history[MAX_HIST];
int history_count = 0;

void save_state() {
    FILE *fp = fopen(STATE_FILE, "w");
    if (!fp) {
        perror("Failed to save state file");
        return;
    }
    for (int i = 0; i < history_count; i++) {
        fprintf(fp, "%s\n", history[i]);
    }
    fclose(fp);
    printf("[State Saved] Written %d history entries to %s\n", history_count, STATE_FILE);
}

void add_history(const char *cmd) {
    if (strlen(cmd) == 0) return;
    if (history_count < MAX_HIST) {
        history[history_count++] = strdup(cmd);
    }
}

void cleanup_resources() {
    save_state();
    for (int i = 0; i < history_count; i++) {
        free(history[i]);
    }
    printf("[Cleanup] Freed history buffers and shut down gracefully.\n");
}

int is_valid_var_name(const char *name) {
    if (!name || (!isalpha(name[0]) && name[0] != '_')) return 0;
    for (int i = 1; name[i] != '\0'; i++) {
        if (!isalnum(name[i]) && name[i] != '_') return 0;
    }
}
```

 Get Help
 Exit

 WriteOut
 Justify

 Read File
 Where is

 Prev Pg
 Next Pg

 Cut Text
 UnCut Tex

UW PICO 5.09

File: export_state.c

```

    fclose(fp);
    printf("[State Saved] Written %d history entries to %s\n", history_count, STATE_FILE);
}

void add_history(const char *cmd) {
    if (strlen(cmd) == 0) return;
    if (history_count < MAX_HIST) {
        history[history_count++] = strdup(cmd);
    }
}

void cleanup_resources() {
    save_state();
    for (int i = 0; i < history_count; i++) {
        free(history[i]);
    }
    printf("[Cleanup] Freed history buffers and shut down gracefully.\n");
}

int is_valid_var_name(const char *name) {
    if (!name || (!isalpha(name[0]) && name[0] != '_')) return 0;
    for (int i = 1; name[i] != '\0'; i++) {
        if (!isalnum(name[i]) && name[i] != '_') return 0;
    }
    return 1;
}

int handle_export(const char *arg) {
    const char *eq_pos = strchr(arg, '=');
    if (eq_pos == NULL) {
        if (!is_valid_var_name(arg)) {
            printf("export: '%s': not a valid identifier\n", arg);
            return 1;
        }
        return 0;
    }

    size_t var_len = eq_pos - arg;
    char var_name[128];
    strncpy(var_name, arg, var_len);
    var_name[var_len] = '\0';

    const char *var_val = eq_pos + 1;

    char clean_val[BUF_SIZE];
    size_t val_len = strlen(var_val);

    if ((var_val[0] == '"' && var_val[val_len - 1] == '"') ||

```

^G Get Help
^X Exit

^O WriteOut
^J Justify

^R Read File
^W Where is

^Y Prev Pg
^V Next Pg

^K Cut Text
^U UnCut Tex

UW PICO 5.09

File: export_state.c

```
    return 1;
}

int handle_export(const char *arg) {
    const char *eq_pos = strchr(arg, '=');
    if (eq_pos == NULL) {
        if (!is_valid_var_name(arg)) {
            printf("export: '%s': not a valid identifier\n", arg);
            return 1;
        }
        return 0;
    }

    size_t var_len = eq_pos - arg;
    char var_name[128];
    strncpy(var_name, arg, var_len);
    var_name[var_len] = '\0';

    const char *var_val = eq_pos + 1;

    char clean_val[BUF_SIZE];
    size_t val_len = strlen(var_val);

    if ((var_val[0] == '"' && var_val[val_len - 1] == '"') ||
        (var_val[0] == '\'' && var_val[val_len - 1] == '\'')) {
        strncpy(clean_val, var_val + 1, val_len - 2);
        clean_val[val_len - 2] = '\0';
    } else {
        strcpy(clean_val, var_val);
    }

    if (!is_valid_var_name(var_name)) {
        printf("export: '%s': not a valid identifier\n", var_name);
        return 1;
    }

    if (setenv(var_name, clean_val, 1) == 0) {
        printf("[Export] Successfully set %s=%s\n", var_name, clean_val);
        return 0;
    } else {
        perror("setenv failed");
        return 1;
    }
}

void test_child_export(const char *var_name) {
    pid_t pid = fork();
    if (pid == 0) {
```

^G Get Help
^X Exit

^O WriteOut
^J Justify

^R Read File
^W Where is

^Y Prev Pg
^V Next Pg

^K Cut Text
^U UnCut Text

UW PICO 5.09

File: export_state.c

```
    char *val = getenv(var_name);
    printf("[Child Process PID %d] Injected %s = %s\n", getpid(), var_name, val ? val : "(null)");
    exit(0);
} else if (pid > 0) {
    waitpid(pid, NULL, 0);
}

int main() {
    char input_buf[BUF_SIZE];
    while (1) {
        char cwd[PATH_MAX];
        if (getcwd(cwd, sizeof(cwd)) == NULL) strcpy(cwd, "shell");
        printf("\n[%s] > ", cwd);

        if (!fgets(input_buf, sizeof(input_buf), stdin)) break;
        input_buf[strcspn(input_buf, "\n")] = 0;

        if (strncmp(input_buf, "exit", 4) == 0 && (input_buf[4] == ' ' || input_buf[4] == '\0')) {
            int exit_code = 0;
            if (strlen(input_buf) > 5) {
                exit_code = atoi(input_buf + 5);
            }
            cleanup_resources();
            printf("Exiting shell with status code %d\n", exit_code);
            exit(exit_code);
        }

        if (strlen(input_buf) == 0) continue;
        add_history(input_buf);

        if (strncmp(input_buf, "export ", 7) == 0) {
            handle_export(input_buf + 7);
        } else if (strncmp(input_buf, "testchild ", 10) == 0) {
            test_child_export(input_buf + 10);
        } else if (strcmp(input_buf, "pwd") == 0) {
            printf("%s\n", cwd);
        } else {
            printf("Command recorded in history. (Use 'export VAR=VAL', 'testchild VAR', 'pwd', or 'exit [code]')\n");
        }
    }

    cleanup_resources();
    return 0;
}
```

^G Get Help
^X Exit

^O WriteOut
^J Justify

^R Read File
^W Where is

^Y Prev Pg
^V Next Pg

^K Cut Text
^U UnCut Text

Shell exited safely.

```
medikondarakesh@Medikondas-MacBook-Air ~ % nano export_state.c
medikondarakesh@Medikondas-MacBook-Air ~ % gcc -g -Wall export_state.c -o export_state
./export_state
```

```
[/Users/medikondarakesh] > export MY_VAR=HelloLinux
[Export] Successfully set MY_VAR=HelloLinux
```

```
[/Users/medikondarakesh] > export 123INVALID=Value
export: '123INVALID': not a valid identifier
```

```
[/Users/medikondarakesh] > testchild MY_VAR
[Child Process PID 4639] Injected MY_VAR = HelloLinux
```

```
[/Users/medikondarakesh] > exit 0
[State Saved] Written 3 history entries to shell_state.txt
[Cleanup] Freed history buffers and shut down gracefully.
Exiting shell with status code 0
medikondarakesh@Medikondas-MacBook-Air ~ % █
```